Phase 1 - Week 1 - Lesson Day 1

Python Refresh for Data Science

Today's Objectives

You already know Python from full-stack development. Today isn't about learning Python from zero - it's about seeing the tools you already know (lists, dicts, functions) through a **data science lens**, and getting comfortable working inside a notebook environment, which is the standard way data scientists write and explore code.

- Recap core Python data types and where they show up in data work
- Understand and write list comprehensions
- Understand and write lambda functions
- Understand why data scientists use Jupyter/Colab notebooks instead of plain .py files

1. Python Data Types - Full Explanation

Definition: A *data type* tells Python what kind of value something is, and what you are allowed to do with it. Python types fall into two groups: **basic (primitive) types**, which hold a single value, and **collection types**, which hold multiple values together.

Basic (Primitive) Types

These hold exactly one value each. You already use these constantly, but here is a quick reference:

Type	Holds	Example
int	Whole numbers	25
float	Decimal numbers	3.14
str	Text	"Hello"
bool	True / False	True
NoneType	No value / empty	None

```
age = 25              # int
price = 3.14           # float
name = "Hello"         # str
is_active = True       # bool
result = None          # NoneType - represents 'nothing here yet'

print(type(age))       # <class 'int'>
print(type(price))     # <class 'float'>
```

Where it matters in data science: every single cell in a dataset is ultimately one of these basic types - a number, a piece of text, a true/false flag, or a missing value (which is often represented as None or NaN). Knowing these cold makes it much easier to understand error messages later, since most bugs come down to a value being the wrong type.

The four types below - **list**, **dictionary**, **tuple**, and **set** - are different: they are **collection types**. Instead of holding one value, each one holds many values (which are often basic types like int or str) grouped together in a specific way.

Collection Types

List

A **list** is an ordered collection of items. "Ordered" means the items keep the position you put them in, and you can access any item using its position (its *index*), starting at 0. Lists are **mutable**, which means you can change them after creating them - add items, remove items, or change an item at a specific position.

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])        # 'apple'  -> first item, index 0
print(fruits[-1])       # 'cherry' -> last item

fruits.append("mango")  # add an item to the end
fruits[1] = "blueberry" # change an item
fruits.remove("apple")  # remove a specific item
print(len(fruits))      # number of items in the list
```

Common list methods: .append() adds one item to the end. .extend() adds multiple items from another list. .remove() deletes the first matching item. .sort() sorts the list in place. .pop() removes and returns the last item (or an item at a given index).

Where it matters in data science: lists are usually the first place raw data lands before you convert it into a NumPy array or a Pandas column, because they are simple and flexible.

Dictionary

A **dictionary** stores data as **key-value pairs** instead of positions. Instead of asking "give me the item at position 0", you ask "give me the value stored under this key". Dictionaries are also mutable, and as of modern Python they keep insertion order, though you should not rely on position the way you do with lists - always look things up by key.

```
student = {"name": "Ali", "age": 25, "course": "Data Science"}

print(student["name"])       # 'Ali'  -> look up by key
student["age"] = 26           # update a value
student["city"] = "Mogadishu" # add a new key-value pair

print(student.keys())         # all the keys
print(student.values())       # all the values
print(student.get("email", "N/A"))  # safe lookup with a default
```

Where it matters in data science: a dictionary is exactly what a single JSON record or a single API response usually looks like - a set of named fields. You will convert lists of dictionaries into full tables (DataFrames) very often.

Tuple

A **tuple** looks like a list (an ordered collection you access by index), but it is **immutable** - once created, you cannot add, remove, or change its items. This makes tuples useful for data that should never accidentally be modified, and they are also slightly faster and use less memory than lists.

```
location = (37.77, -122.41)   # latitude, longitude

print(location[0])            # 37.77
# location[0] = 40.0          # this would raise an error - tuples cannot be changed

lat, lon = location           # 'unpacking' a tuple into two variables
print(lat, lon)
```

Where it matters in data science: tuples are commonly used for fixed pairs/groups of values such as coordinates, image dimensions (width, height), or fixed configuration settings that a function should not be able to accidentally alter.

Set

A **set** is an unordered collection of **unique** items - duplicates are automatically removed, and items have no fixed position, so you cannot access a set by index. Sets are mutable (you can add or remove items), and they are optimized for very fast membership checks ("is this item in here?") and for comparing collections against each other.

```
ids = {101, 102, 102, 103}    # duplicate 102 is automatically dropped
print(ids)                    # {101, 102, 103}

ids.add(104)                  # add an item
print(103 in ids)             # True -> fast membership check

a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)                  # intersection -> {2, 3}
print(a | b)                  # union -> {1, 2, 3, 4}
```

Where it matters in data science: sets are the standard tool for quickly removing duplicate records or checking overlap between two groups of data (for example, comparing two lists of customer IDs to find which customers appear in both).

Quick Comparison

Type	Ordered?	Changeable?	Duplicates allowed?	Access by
List	Yes	Yes (mutable)	Yes	Index (position)
Dictionary	Yes (insertion order)	Yes (mutable)	Keys must be unique	Key

Tuple	Yes	No (immutable)	Yes	Index (position)
Set	No	Yes (mutable)	No (auto-removed)	Membership check (in)

2. List Comprehensions

Definition: A *list comprehension* is a compact way to build a new list by applying an expression to every item in an existing iterable (like a list or range), optionally filtering items - all written in a single line instead of a multi-line loop.

The traditional way, using a loop:

```
squares = []
for x in range(10):
    squares.append(x**2)
```

The list comprehension way (identical result, one line):

```
squares = [x**2 for x in range(10)]
```

Why it matters in data science: you'll use this constantly to quickly transform raw data before it goes into a model - for example, converting a list of raw prices into a list of prices with tax applied, in one clean line.

You can also filter while building the list:

```
evens = [x for x in range(20) if x % 2 == 0]
```

Read this as: *"give me x, for every x in range(20), but only if x is divisible by 2."* The **if** at the end is the filter condition.

3. Lambda Functions

Definition: A *lambda function* is a small, unnamed (anonymous) function written in a single line, typically used for short, throwaway operations where defining a full function with **def** would feel like overkill.

```
square = lambda x: x**2
print(square(5))   # 25
```

This is functionally identical to:

```
def square(x):
    return x**2
```

Why it matters in data science: lambdas are used heavily with **map()**, **filter()**, and later with Pandas' **.apply()** to transform columns of data on the fly, without writing a separate named function for every small transformation.

map() - applies a function to every item in a list:

```
numbers = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, numbers))
# [2, 4, 6, 8]
```

filter() - keeps only items where the function returns True:

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4, 6]
```

4. Jupyter Notebook / Google Colab

Definition: These are *interactive coding environments* where you write and run code in small chunks called *cells*, seeing the output immediately below each cell - instead of running an entire .py file at once from top to bottom.

Why data scientists use this instead of plain .py files: data work is exploratory. You're constantly checking what a dataset looks like, testing one transformation, plotting a quick chart, then deciding the next step based on what you saw. Re-running an entire script every time (as is normal in software engineering) would be painfully slow and wasteful.

Since you're both coming from full-stack development, think of a notebook like a live REPL that remembers your variables across cells, and lets you mix code, output, charts, and written notes in a single scrollable document. Google Colab (colab.research.google.com) is a free, browser-based version - no installation needed, which is what we'll use for this course.

Today's Exercise

Open the accompanying notebook file **day-1-python-refresh.ipynb** and complete the exercises below inside it. Each one maps directly to a concept covered above - read the explanation of what's being asked, then write the code yourself in the notebook before checking back here.

Part A - List Comprehensions (6 questions)

1. Squares from 1-10

Build a list containing the square of every number from 1 to 10. This is the exact pattern shown in the lesson - one expression, one loop variable, no filter needed.

2. Filter even numbers (1-50)

Loop through numbers 1 to 50 and keep only the ones divisible by 2. This uses the comprehension-with-a-filter pattern from the lesson (the trailing **if**).

3. Word lengths

Given **["data", "science", "AI", "python"]**, build a new list containing the length of each word. Think about which built-in function returns the length of a string.

4. Celsius to Fahrenheit

Given **[0, 10, 20, 30]**, build a new list converting each value using the formula (C * 9/5) + 32. This is a pure transformation, no filtering needed.

5. Words starting with a vowel

Given **["apple", "banana", "orange", "grape", "egg"]**, keep only the words whose first letter is a vowel (a, e, i, o, u). You will need to check the first character of each word inside the **if** condition.

6. Number/even-flag pairs

Build a list of tuples (number, is_even) for numbers 1 to 10, where is_even is True or False. For example, the first pair should be (1, False) since 1 is odd. This combines a tuple expression with the modulo operator (%).

Part B - Lambda Functions (4 questions)

7. Cube with map()

Write a lambda function that cubes a number (x**3), then use map() to apply it to [1, 2, 3, 4] and convert the result to a list.

8. Remove negatives with filter()

Write a lambda that returns True only for positive numbers, then use filter() to remove all negative numbers from [-5, 3, -2, 8, -1, 10].

9. Long words with filter()

Write a lambda that checks whether a string has more than 5 characters, then use filter() to keep only the long words from ["cat", "elephant", "dog", "python", "ai"].

10. Uppercase with map()

Write a lambda that converts a string to uppercase (hint: use the .upper() string method inside the lambda), then use map() to apply it to ["ali", "halimo", "data", "science"].

How to Approach It

For every exercise: first identify what the **loop variable** is, what the **expression** (the transformation) is, and whether a **filter condition** is needed. If you get stuck, write the slow version first (a normal for-loop with .append()), get it working, then compress it into a one-line comprehension or lambda - that's exactly how experienced data scientists debug these when they don't come out right the first time.

Once you and Halimo have both completed the exercises in the notebook, bring your answers back to the lesson chat and we'll review them together before moving on to Lesson Day 2 (NumPy Basics).